



Tiếng Việt



Trang chủ Class & Structure Mảng liên tiếp (Array) và Danh sách liên kết (Linked List) Cây tìm kiếm nhị phân (Binary Search Tree) Đồ thị (Graph) Thuật toán tìm kiếm (Search Algorithms) Băm xâu (String Hashing) DFS/BFS

Băm xâu (String Hashing)

Nội dung

- 0. Tại sao ta cần băm xâu?
- 1. Thử thiết kế một hàm hash đơn giản
- 2. Băm đa thức (polynomial hash)
- 3. Tính hash của một đoạn nhanh — Hash tiền tố
- 4. Hash của một đoạn — $\mathcal{O}(1)$
- 5. Hasher: gói tất cả vào class
- 6. So sánh hai đoạn xâu — $\mathcal{O}(1)$
- 7. Tìm xuất hiện của pattern
- 8. Đếm số xâu con phân biệt
- 9. Đụng độ & Double Hashing
- 10. Bẫy thường gặp
- 11. Tổng kết

0. Tại sao ta cần băm xâu?

Hãy thử **một bài rất thực tế**:

“

Cho xâu s độ dài n và Q truy vấn, mỗi truy vấn cho 4 số (l_1, r_1, l_2, r_2) . Hỏi đoạn $s[l_1..r_1]$ có bằng đoạn $s[l_2..r_2]$ không?

Cách "ngây thơ": với mỗi truy vấn, so sánh **từng ký tự** — mất $\mathcal{O}(L)$ với L là độ dài đoạn. Tổng $\mathcal{O}(Q \cdot L)$. Nếu $n = Q = L = 10^5$, tổng $\approx 10^{10}$ phép — **quá chậm**.

Cách thông minh: nếu ta gán cho mỗi xâu **một con số** sao cho:

- Xâu giống nhau $\rightarrow$ cùng số.
- Xâu khác nhau $\rightarrow$ (hầu như chắc chắn) khác số.

Thì so sánh 2 đoạn chỉ còn **so sánh 2 số** — $\mathcal{O}(1)$! Tổng $\mathcal{O}(n + Q)$ — siêu nhanh.

Con số đó gọi là **hash** (hay "dấu vân tay") của xâu, và kỹ thuật để tính nhanh hash của mọi đoạn gọi là **string hashing**.



Tiếng Việt



những kỹ thuật **lời nhất** ở mức Bạc.

1. Thử thiết kế một hàm hash đơn giản

Ý tưởng đầu tiên: cộng mã ASCII của mọi ký tự lại.

$$\text{hash}_{\text{ngây thơ}}(\text{"abc"}) = 1 + 2 + 3 = 6$$

(với mã $a \rightarrow 1, b \rightarrow 2, c \rightarrow 3$).

Có vấn đề gì? Hãy thử `"bca"`:

$$\text{hash}_{\text{ngây thơ}}(\text{"bca"}) = 2 + 3 + 1 = 6$$

Cùng số! Hai xâu khác nhau lại ra cùng hash — **đụng độ** (collision). Phép cộng **không nhớ thứ tự** — đây là điểm chết.

Ta cần một công thức **để vị trí của ký tự cũng quan trọng**. Cách tự nhiên nhất: nhân với **lũy thừa của một cơ số** — giống cách viết một con số trong hệ thập phân.

2. Băm đa thức (polynomial hash)

Ý tưởng: "đọc" xâu như một **con số viết trong cơ số B** .

Ví dụ với $B = 10$ (cơ số quen thuộc nhất — hệ thập phân): mã $a \rightarrow 1, b \rightarrow 2, c \rightarrow 3, d \rightarrow 4$:

$$H(\text{"abc"}) = 1 \cdot 10^2 + 2 \cdot 10^1 + 3 \cdot 10^0 = 100 + 20 + 3 = \boxed{123}$$

Đọc lên đúng là... **123**! Tức là xâu `"abc"` được "đọc" thành con số 123 trong hệ thập phân.

Tương tự:

$$H(\text{"bca"}) = 2 \cdot 100 + 3 \cdot 10 + 1 = 231 \neq 123$$

Khác nhau! Vị trí ký tự đã ảnh hưởng đến giá trị. **Không còn đụng độ ngu xuẩn** như cách cộng.

Công thức tổng quát: với xâu $s = s_0 s_1 \dots s_{n-1}$ (mỗi s_i là một số nguyên dương — mã ký tự):

$$H(s) = (s_0 \cdot B^{n-1} + s_1 \cdot B^{n-2} + \dots + s_{n-1} \cdot B^0) \bmod M$$



Tiếng Việt

ngẫu nhiên. Phổ biến: $B = 31, 99, 101, 311$.

- M (**mod**): vì con số này có thể **rất lớn** (xâu dài $10^5 \rightarrow$ số có 10^5 chữ số!), ta lấy modulo M để vừa kiểu `long long`. Phổ biến: $M = 10^9 + 7$ hoặc $10^9 + 9$ (số nguyên tố lớn).

Ví dụ với $B = 31$ (thực tế hơn): với `a→1, b→2, c→3` :

$$H(\text{"abc"}) = 1 \cdot 31^2 + 2 \cdot 31 + 3 = 961 + 62 + 3 = 1026$$

“

Lưu ý quan trọng: Cộng 1 vào mã ký tự (`s[i] - 'a' + 1` thay vì `s[i] - 'a'`). Nếu không, `'a' → 0`, dẫn đến $H(\text{"a"}) = 0$, $H(\text{"aa"}) = 0$, $H(\text{"aaa"}) = 0$ — tất cả các xâu chỉ chứa `'a'` đều có hash 0 $\rightarrow$ đụng độ hàng loạt.

3. Tính hash của một đoạn nhanh — Hash tiền tố

Tới đây ta đã hash được **cả xâu** trong $O(n)$. Nhưng bài toán hỏi hash của **đoạn bất kỳ** $s[l..r]$ — nếu cứ tính lại từ đầu thì vẫn $O(n)$ /truy vấn, không nhanh hơn cách ngây thơ!

Bí kíp: dùng **hash tiền tố** — y như prefix sum (đã học ở khoá Đồng), nhưng cho hash. Đặt:

$$h[i] = H(s[0..i-1]) = (s_0 \cdot B^{i-1} + s_1 \cdot B^{i-2} + \dots + s_{i-1}) \bmod M$$

quy ước $h[0] = 0$ (hash của xâu rỗng).

Xây dựng từng bước — với $s = \text{"abc"}$, $B = 10$, mã `a→1, b→2, c→3` :

I	$H[I]$	Ý NGHĨA
0	0	hash của <code>""</code>
1	$0 \cdot 10 + 1 = 1$	hash của <code>"a"</code>
2	$1 \cdot 10 + 2 = 12$	hash của <code>"ab"</code>
3	$12 \cdot 10 + 3 = 123$	hash của <code>"abc"</code>

Công thức truy hồi: $h[i+1] = h[i] \cdot B + s_i$. (Đẩy mọi mũ lên 1 bậc rồi thêm ký tự mới ở mũ B^0 .) Đây là bước **xây prefix hash**.



Tiếng Việt



Có $h[]$ rồi, làm sao lấy hash của đoạn $s[l..r]$?

Quan sát: với $s = \text{"abcde"}$, $B = 10$:

- $h[5] = 12345$ (đọc "abcde")
- $h[2] = 12$ (đọc "ab")

Muốn lấy "cde" ($= 345$), ta thấy: $12345 - 12000 = 345$. Mà $12000 = 12 \cdot 1000 = h[2] \cdot 10^3$. Số 3 chính là **độ dài đoạn**!

Công thức tổng quát — đoạn $s[l..r]$ có độ dài $L = r - l + 1$:

$$\text{hash}(l, r) = (h[r + 1] - h[l] \cdot B^L) \bmod M$$

Tại sao công thức đúng? $h[r + 1]$ chứa toàn bộ ký tự từ vị trí 0 tới r , mũ cao nhất là B^r . $h[l]$ chứa ký tự 0 tới $l - 1$, mũ cao nhất là B^{l-1} . Nhân $h[l]$ với B^L "đẩy" lên cùng thang với $h[r + 1]$, trừ đi triệt tiêu phần đầu — còn lại chính là phần giữa.

Kiểm tra với "abcde", lấy "bcd" ($l=1, r=3, L=3$):

$$\text{hash}(1, 3) = h[4] - h[1] \cdot 10^3 = 1234 - 1 \cdot 1000 = 234 = \text{"bcd"} \checkmark$$

Tiền tính $\text{pw}[i] = B^i \bmod M$ ngay khi build, để mỗi truy vấn chỉ tốn $O(1)$.

5. Hasher: gói tất cả vào class

Như đã giới thiệu ở **bài 1**, để tái sử dụng dễ dàng ta gói toàn bộ vào một `struct` / `class`. Constructor xây luôn $h[]$ và $\text{pw}[]$, hàm `get(l, r)` trả hash của đoạn.

C++

Python



```
static const long long B = 131;
static const long long M = 1000000007;
vector<long long> h, pw;

Hasher(const string& s) {
    int n = s.size();
    h.assign(n + 1, 0);
    pw.assign(n + 1, 1);
    for (int i = 0; i < n; i++) {
        h[i + 1] = (h[i] * B + (s[i] - 'a' + 1)) % M;
        pw[i + 1] = pw[i] * B % M;
    }
}

// hash của s[l..r] (0-indexed, hai đầu bao gồm)
long long get(int l, int r) {
    long long L = r - l + 1;
    long long x = (h[r + 1] - h[l] * pw[L]) % M;
    return (x % M + M) % M;    // tránh số âm sau %
}

};

// Sử dụng:
Hasher H(s);
if (H.get(l1, r1) == H.get(l2, r2)) { /* hai đoạn (gần như chắc chắn) bằng nhau */ }
```



```

B = 131
M = 1_000_000_007

def __init__(self, s):
    n = len(s)
    self.h = [0] * (n + 1)
    self.pw = [1] * (n + 1)
    for i, c in enumerate(s):
        self.h[i + 1] = (self.h[i] * self.B + (ord(c) - ord('a') + 1))
        % self.M
        self.pw[i + 1] = self.pw[i] * self.B % self.M

def get(self, l, r):
    L = r - l + 1
    x = (self.h[r + 1] - self.h[l] * self.pw[L]) % self.M
    return x    # Python % luôn cho kết quả không âm

```

Quan trọng: trong C++, sau phép trừ $h[r+1] - h[l] * pw[L]$ có thể ra **số âm** (vì $h[l] * pw[L]$ có thể lớn hơn $h[r+1]$ modulo M). Phải $((x \% M) + M) \% M$ để đưa về $[0, M)$. Python không gặp lỗi này vì toán tử `%` luôn cho kết quả không âm.

6. So sánh hai đoạn xâu — $O(1)$

Đây là ứng dụng phổ biến nhất — chính là bài toán đặt ra ở mục 0:

C++ Python

```

Hasher H(s);
// s[l1..r1] có bằng s[l2..r2] không?
bool equal_segment(int l1, int r1, int l2, int r2) {
    if (r1 - l1 != r2 - l2) return false;    // độ dài khác → khác
    return H.get(l1, r1) == H.get(l2, r2);
}

```



Tiếng Việt



```

if r1 - l1 != r2 - l2:
    return False
return H.get(l1, r1) == H.get(l2, r2)

```

Trace với $s = \text{"abacaba"}$: hỏi $s[0..2]$ ($= \text{"aba"}$) có bằng $s[4..6]$ ($= \text{"aba"}$) không?

- $H(s[0..2]) = \text{hash của "aba"}$
- $H(s[4..6]) = \text{hash của "aba"}$
- Bằng $\rightarrow$ in **"YES"**.

Còn $s[0..1]$ ($= \text{"ab"}$) so với $s[2..3]$ ($= \text{"ac"}$)? Hash chắc chắn khác $\rightarrow$ in **"NO"**.

Bài học: chỉ tốn $O(1)$ cho mỗi truy vấn, sau khi đã build Hasher trong $O(n)$.

7. Tìm xuất hiện của pattern

Đề: Cho text T độ dài n và pattern P độ dài m . Đếm số lần P xuất hiện trong T .

Ý tưởng: Tính hash của P một lần (gọi là **target**), rồi **trượt cửa sổ độ dài m** trên T (vị trí bắt đầu $i = 0, 1, \dots, n - m$). Mỗi cửa sổ so sánh hash với **target** trong $O(1)$.

Trace với $T = \text{"ababab"}$, $P = \text{"ab"}$ ($n = 6, m = 2$):

I	ĐOẠN $T[I..I+1]$	BẰNG "AB" ?
0	"ab"	✓
1	"ba"	X
2	"ab"	✓
3	"ba"	X
4	"ab"	✓

Đáp án: **3 lần xuất hiện.**



```
int count_occurrences(const string& T, const string& P) {
    int n = T.size(), m = P.size();
    if (m > n) return 0;
    Hasher HT(T), HP(P);
    long long target = HP.get(0, m - 1);
    int cnt = 0;
    for (int i = 0; i + m - 1 < n; i++) {
        if (HT.get(i, i + m - 1) == target) cnt++;
    }
    return cnt;
}
```

```
def count_occurrences(T, P):
    n, m = len(T), len(P)
    if m > n:
        return 0
    HT, HP = Hasher(T), Hasher(P)
    target = HP.get(0, m - 1)
    return sum(1 for i in range(n - m + 1) if HT.get(i, i + m - 1) ==
target)
```

Tổng phức tạp: $O(n + m)$. Nhanh hơn nhiều so với cài KMP!

8. Đếm số xâu con phân biệt

Đề: Cho xâu s , đếm số xâu con (substring) phân biệt.

Ví dụ nhỏ: $s = \text{"aba"}$. Các xâu con là:

- Độ dài 1: $\text{"a"}, \text{"b"}, \text{"a"} \rightarrow$ phân biệt: $\{\text{"a"}, \text{"b"}\} \rightarrow 2$.
- Độ dài 2: $\text{"ab"}, \text{"ba"} \rightarrow$ phân biệt: $\{\text{"ab"}, \text{"ba"}\} \rightarrow 2$.
- Độ dài 3: $\text{"aba"} \rightarrow 1$.

Tổng: $2 + 2 + 1 = 5$ xâu con phân biệt.



tử trong `set` là số xâu con **phân biệt** có độ dài L . Cộng lại là đáp án.

C++

```
long long count_distinct_substrings(const string& s) {
    int n = s.size();
    Hasher H(s);
    long long total = 0;
    for (int L = 1; L <= n; L++) {
        set<long long> seen;
        for (int i = 0; i + L - 1 < n; i++) {
            seen.insert(H.get(i, i + L - 1));
        }
        total += seen.size();
    }
    return total;
}
```

Phức tạp: $O(n^2 \log n)$ vì insert vào `set`. Đủ tốt với $n \leq 5000$. Với n lớn hơn cần suffix array (khoá Vàng).

9. Đụng độ & Double Hashing

Hash có thể **đụng độ** (collision): hai xâu khác nhau ra cùng hash. Xác suất 1 cặp đụng độ $\approx 1/M$. Nhưng nhiều bài lại so sánh **rất nhiều cặp**:

- Bài có $Q = 10^5$ truy vấn so sánh 2 đoạn $\rightarrow$ khoảng 10^5 cặp $\rightarrow$ xác suất sai $\approx 10^{-4}$ — thường an toàn.
- Bài "đếm xâu con phân biệt" hoặc các bài duyệt **mọi cặp đoạn**: số cặp lên tới $O(n^2) \approx 10^{10}$ với $n = 10^5 \rightarrow$ xác suất sai $\approx 10^{10}/10^9 = 10$ — **gần như chắc chắn sai!**

Đó là chưa kể **anti-hash test** trên judge công khai B, M (như Codeforces): người ra đề có thể chuẩn bị input ép đụng độ — xác suất sai = 1.

Giảm rủi ro: dùng **hai cặp** (B_1, M_1) và (B_2, M_2) khác nhau. Hai đoạn bằng nhau $\Leftrightarrow$ **cả hai** hash bằng. Xác suất 1 cặp đụng độ $\approx 1/(M_1 \cdot M_2) \approx 10^{-18}$ — kể cả với 10^{10} cặp vẫn an toàn.



```
// Lưu hash dạng pair<long long, long long>, so sánh bằng ==
struct Hasher2 {
    vector<long long> h1, pw1, h2, pw2;
    static const long long B1 = 131, M1 = 1000000007;
    static const long long B2 = 137, M2 = 998244353;
    // build & get tương tự, chỉ là 2 lần song song
};
```

Trong CP, một hash thường là đủ — chỉ thêm double hash khi bài là CF Round có anti-hash.

10. Bẫy thường gặp

- **Quên chuẩn hoá modulo sau phép trừ** (C++): luôn dùng $((x \% M) + M) \% M$.
- **Hash của xâu rỗng = 0**: cẩn thận khi $l > r$ hoặc khi độ dài bằng 0.
- **Quên cộng 1 vào mã ký tự**: $s[i] - 'a'$ cho $'a' \rightarrow 0$, khiến $"a"$ và $"aa"$ có cùng hash 0.
- **Anti-hash trên Codeforces**: với B và M cố định công khai, người ra đề có thể chuẩn bị test đụng độ. Cách phòng: chọn B random ở runtime (ví dụ `mt19937 rng(chrono::steady_clock::now().time_since_epoch().count()); B = rng() % 1000 + 256;`), hoặc dùng double hash. Tránh `rand()` — chu kỳ ngắn và dễ đoán.
- **Dùng `unordered_map<long long, ...>`** với key là hash: anh em Codeforces có thể "hack" bằng input gây nhiều đụng độ. Dùng `map` cho an toàn.

11. Tổng kết

THAO TÁC	PHỨC TẠP
Build Hasher	$O(n)$
So sánh 2 đoạn	$O(1)$
Tìm pattern trong text	$O(n + m)$



Tiếng Việt



Đếm xâu con phân biệt ($n \leq 5000$)	$O(n^2 \log n)$
---	-----------------

String hashing là **công cụ Swiss Army Knife** cho mọi bài liên quan đến so sánh đoạn xâu. Học thuộc một class `Hasher` và mang theo trong template — bạn sẽ dùng nó **rất nhiều!**

Bài tập

Rút gọn xâu	0 / 15
Ghép xâu	0 / 15
CSES - String Matching Khớp xâu	0 / 20
CSES - Finding Borders Tìm biên	0 / 25
CSES - Finding Periods Tìm chu kì	0 / 25
CSES - Pattern Positions Vị trí xâu con	0 / 25
Ước chung của chuỗi	0 / 25
Xâu con lặp	0 / 25
Tạo palindrome	0 / 25
CSES - Longest Palindrome Xâu đối xứng dài nhất	0 / 30
CSES - Repeating Substring Xâu con lặp	0 / 30

proudly powered by **DMOJ** | [developed by LQDJudge team](#) Tiếng Việt (vi) ▼